

LAPORAN PRAKTIKUM 3
STRUKTUR DATA STACK

Disusun Oleh:

Dinda Amelia

(2511531020)

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Muhammad Galid Avero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG 2026

KATA PENGANTAR

Segala puji bagi Allah SWT yang telah memberikan kesempatan serta kemudahan kepada penulis untuk dapat menyelesaikan Laporan Praktikum pada mata kuliah Struktur Data, sehingga Laporan Praktikum ini dapat dikumpulkan dengan tepat waktu. Atas rahmat dan karunianya Laporan Praktikum dapat terselesaikan dengan baik. Sholawat serta salam juga penulis sampaikan kepada baginda tercinta yaitu Nabi Muhammad SAW, yang kita nantikan syafa'atnya di akhirat nanti. Laporan Praktikum ini bertujuan untuk menambah wawasan para pembaca untuk lebih memperdalam ilmu yang ada pada makalah ini.

Laporan pratikum ini disusun sebagai bentuk penanggung jawaban atas pelaksanaan kegiatan pratikum mata kuliah struktur data yang membahas tentang Stack di Java Eclipse. Dalam praktikum ini, penulis tidak hanya belajar tentang Penyusunan laporan ini bertujuan untuk memberikan pemahaman mengenai memberikan keterampilan praktis dalam mengelola data sementara.

Dalam penyusunan Laporan Praktikum ini, penulis mengalami banyak kesulitan dan penulis menyadari bahwa Laporan Praktikum ini masih jauh dari kesempurnaan. Untuk itu, penulis sangat mengharapkan kritik dan saran yang membangun demi kesempurnaan pada Laporan Praktikum ini.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I	1
PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan	2
1.3 Manfaat Praktikum.....	2
BAB II	3
PEMBAHASAN	3
2.1 Dasar Teori.....	3
2.2 Langkah kerja	4
2.2.1 Buka eclipse.....	4
2.2.2 Membuat Java Project	4
2.2.3 Class 1 “latihanStack_2511531020”	5
2.2.4 Class 2 “StackPostfix_2511531020”	8
2.2.5 Class 3 “NilaiMaksimum_2511531020”	11
2.2.6 Class 4 “stackArray_2511531020”	15
2.2.7 Class 5 “stackArrayDriver_2511531020”	18
2.2.8 Class 6 “Siswa_2511531020”.....	21
2.3 Tugas Praktikum.....	25
2.3.1 Class Website_2511531020	25
2.3.2 Class Browser_2511531020	28
BAB III	33
KESIMPULAN	33
DAFTAR PUSTAKA	34

BAB I

PENDAHULUAN

1.1 Latar Belakang

Stack merupakan salah satu struktur data fundamental dalam ilmu komputer yang berperan penting dalam pengelolaan data sementara. Konsep dasar Stack adalah *Last In First Out (LIFO)*, yaitu elemen terakhir yang dimasukkan akan menjadi elemen pertama yang dikeluarkan. Prinsip ini menjadikan Stack sangat relevan dalam berbagai aplikasi pemrograman, terutama ketika diperlukan kontrol terhadap urutan eksekusi data.

Dalam konteks sejarah perkembangan struktur data, Stack muncul sebagai solusi untuk kebutuhan pengelolaan memori dan eksekusi instruksi pada sistem komputer. Misalnya, dalam proses pemanggilan fungsi (*function call*), Stack digunakan untuk menyimpan alamat pengembalian sehingga program dapat kembali ke titik eksekusi sebelumnya setelah fungsi selesai dijalankan. Hal ini menjadikan Stack sebagai komponen vital dalam implementasi rekursi, evaluasi ekspresi matematika, serta mekanisme *undo/redo* pada perangkat lunak.

Di Java, Stack diimplementasikan melalui kelas `java.util.Stack` yang merupakan turunan dari `Vector`. Kelas ini menyediakan berbagai metode bawaan seperti `push()`, `pop()`, `peek()`, dan `empty()` yang memudahkan programmer dalam mengelola data. Selain itu, pemahaman Stack juga dapat diperkuat melalui implementasi manual menggunakan array atau generics, sehingga mahasiswa dapat memahami mekanisme internal dari operasi dasar Stack.

Secara praktis, latar belakang pembelajaran Stack dalam praktikum bertujuan untuk memberikan pemahaman mendalam mengenai cara kerja struktur data ini, melatih keterampilan problem solving, serta membekali mahasiswa dengan dasar yang kuat untuk mempelajari algoritma lebih kompleks seperti *Depth First Search (DFS)* pada graf dan evaluasi ekspresi postfix. Dengan demikian, Stack tidak hanya menjadi konsep teoritis, tetapi juga fondasi penting dalam pengembangan perangkat lunak modern.

1.2 Tujuan

1. Memahami konsep dasar struktur data Stack dengan prinsip *Last In First Out (LIFO)*.
2. Melatih penggunaan operasi fundamental stack seperti push, pop, peek, dan isEmpty.
3. Mengimplementasikan stack dalam berbagai bentuk: menggunakan array, ArrayList, maupun class bawaan Java (`java.util.Stack`).
4. Menerapkan konsep OOP seperti enkapsulasi, abstraksi, dan reusabilitas dalam program berbasis stack.
5. Menunjukkan aplikasi nyata stack dalam kehidupan sehari-hari, misalnya simulasi browser history dan evaluasi ekspresi postfix.

1.3 Manfaat Praktikum

1. Memberikan keterampilan praktis dalam mengelola data menggunakan struktur stack.
2. Membantu mahasiswa memahami penerapan prinsip LIFO dalam konteks pemrograman dan kasus nyata.
3. Membiasakan mahasiswa dengan aturan penamaan, kerapihan kode, serta penanganan kondisi error (*overflow* dan *underflow*).
4. Menjadi dasar untuk mempelajari algoritma dan struktur data yang lebih kompleks di masa depan.
5. Melatih kemampuan problem solving, abstraksi, dan penerapan konsep OOP dalam pengembangan perangkat lunak.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Struktur data Stack merupakan salah satu konsep fundamental dalam ilmu komputer yang bekerja dengan prinsip *Last In First Out (LIFO)*, yaitu elemen terakhir yang dimasukkan ke dalam tumpukan akan menjadi elemen pertama yang dikeluarkan. Konsep ini banyak digunakan dalam berbagai aplikasi, seperti pengelolaan memori, pemanggilan fungsi secara rekursif, evaluasi ekspresi matematika, serta simulasi riwayat pada browser.

Dalam bahasa pemrograman Java, stack dapat diimplementasikan dengan beberapa cara, antara lain menggunakan class bawaan `java.util.Stack`, menggunakan array statis, maupun dengan memanfaatkan struktur dinamis seperti `ArrayList`. Setiap implementasi memiliki kelebihan dan kekurangan: array memberikan kontrol penuh terhadap indeks tetapi terbatas pada ukuran yang ditentukan, sedangkan `ArrayList` dan `Stack` bawaan Java lebih fleksibel karena dapat menyesuaikan ukuran secara dinamis.

Operasi dasar pada stack meliputi:

- Push: menambahkan elemen baru ke bagian atas stack.
- Pop: menghapus elemen teratas dari stack.
- Peek: melihat elemen teratas tanpa menghapusnya.
- isEmpty: memeriksa apakah stack kosong.

Selain itu, dalam konteks pemrograman berorientasi objek (OOP), implementasi stack sering dikaitkan dengan konsep enkapsulasi (menyembunyikan detail internal melalui metode khusus), abstraksi (menyediakan antarmuka sederhana untuk operasi kompleks), serta reusabilitas (kelas ADT dapat digunakan kembali dalam berbagai simulasi).

2.2 Langkah kerja

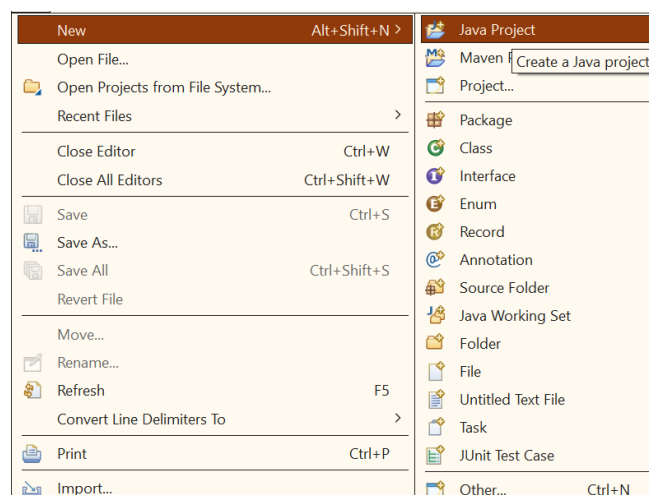
2.2.1 Buka eclipse



Gambar 2.1 Buka eclipse jalankan

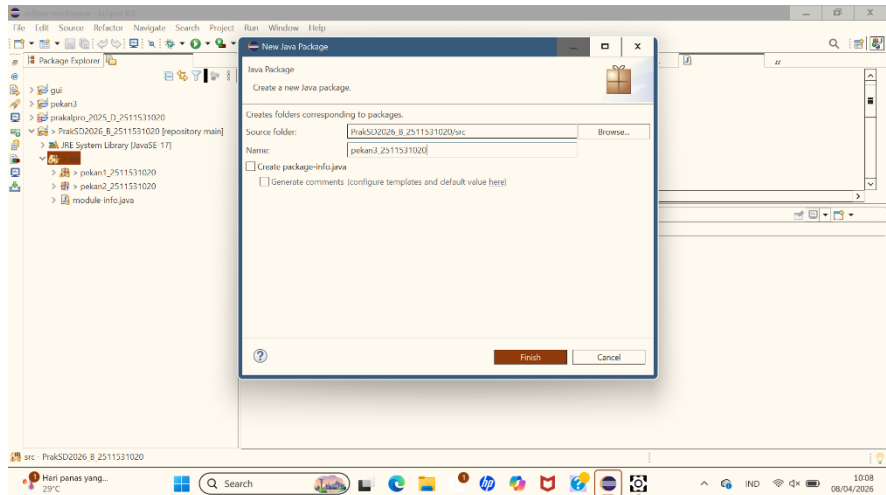
2.2.2 Membuat Java Project

1. Klik file
2. Klik java project



Gambar 2.2 Java Project

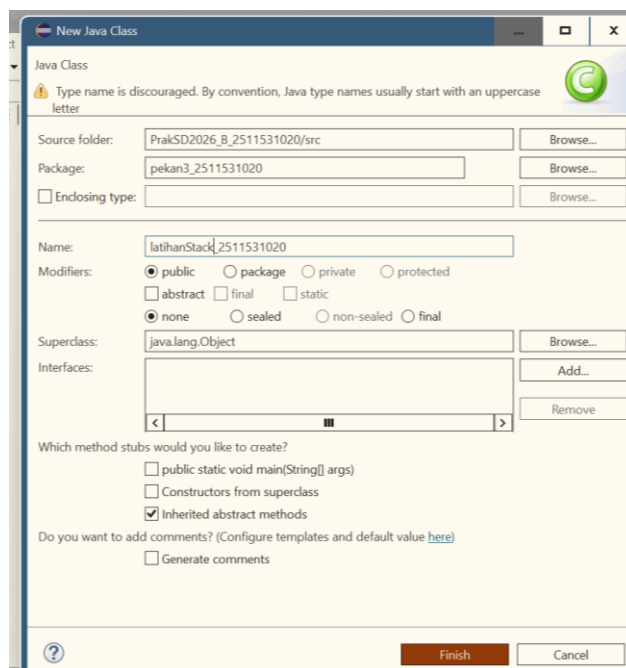
3. Buat nama project



Gambar 2.3 Nama Project

2.2.3 Class 1 “latihanStack_2511531020”

1. Membuat package pekan3_2511531020 dan class latihanStack_2511531020



Gambar 2.4 Membuat Nama package dan class

2. Klik Finish
3. Kode Program


```

1 package pekan3_2511531020;
2
3 import java.util.Stack;
4
5 public class latihanStack_2511531020 {
6     public static void main(String[] args) {
7         Stack<Integer> s = new Stack<Integer> ();
8         s.push(42);
9         s.push(-3);
10        s.push(17);
11        System.out.println("nilai Stack= " + s);
12        System.out.println("nilai pop = " + s.pop());
13        System.out.println("nilai stack setelah pop= "+s);
14        System.out.println("nilai peek"+s.peek());
15        System.out.println("nilai stack setelah peek= "+s);
16    }
17 }
18
19 }
20

```

Gambar 2.5 Class 1 “latihanStack_2511531020”

4. Analisis Langkah

a. Tujuan Program

- Memahami penggunaan kelas Stack di Java.
- Melakukan operasi penambahan elemen (*push*), penghapusan elemen (*pop*), serta melihat elemen teratas (*peek*).
- Menunjukkan prinsip kerja Stack dengan konsep *Last In First Out (LIFO)*.

b. Struktur Program

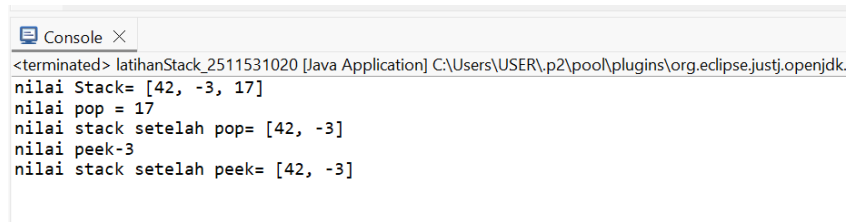
- Package: pekan3_2511531020 → menunjukkan program berada dalam paket tertentu.
- Class: latihanStack_2511531020 → berfungsi sebagai kelas utama untuk menjalankan program.
- Method utama: public static void main(String[] args) → titik masuk eksekusi program.

c. Proses Eksekusi

- Membuat objek Stack<Integer> s = new Stack<Integer>(); untuk menampung data bertipe integer.
- Menambahkan elemen dengan s.push(42);, s.push(-3);, dan s.push(17);.

- Setelah operasi push, isi stack adalah [42, -3, 17] (17 berada di atas).
- d. Menampilkan Nilai Stack
- `System.out.println("nilai Stack= " + s);` → mencetak seluruh isi stack.
 - Output: nilai Stack= [42, -3, 17].
- e. Operasi Pop (Menghapus Elemen Teratas)
- `System.out.println("nilai pop = " + s.pop());` → menghapus elemen teratas (17) dan menampilkannya.
 - Isi stack setelah pop: [42, -3].
 - Output:
 - nilai pop = 17
 - nilai stack setelah pop= [42, -3].
- f. Operasi Peek (Melihat Elemen Teratas)
- `System.out.println("nilai peek" + s.peek());` → menampilkan elemen teratas tanpa menghapusnya (yaitu -3).
 - Isi stack tetap [42, -3].
 - Output:
 - nilai peek-3
 - nilai stack setelah peek= [42, -3].
- g. Konsep OOP dan Struktur Data
- Encapsulation: Stack dibuat sebagai objek dengan konstruktor, menyembunyikan detail implementasi internal.
 - LIFO Principle: Elemen terakhir yang masuk (17) adalah yang pertama keluar saat pop().
 - Method Utility: `push()`, `pop()`, `peek()`, dan `empty()` menunjukkan fungsi bawaan untuk manipulasi data.
 - Abstraksi: Programmer tidak perlu tahu detail implementasi internal, cukup menggunakan metode yang tersedia.

5. Output

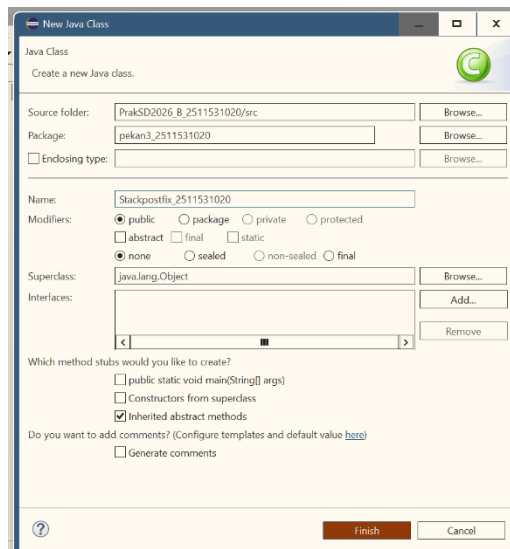


```
<terminated> latihanStack_2511531020 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.openjdk.  
nilai Stack= [42, -3, 17]  
nilai pop = 17  
nilai stack setelah pop= [42, -3]  
nilai peek=3  
nilai stack setelah peek= [42, -3]
```

Gambar 2.6 Output

2.2.4 Class 2 “StackPostfix_2511531020”

1. Membuat package pekan3_2511531020 dan class StackPostfix_2511531020



Gambar 2.7 Membuat Nama package dan class

2. Klik Finish

3. Kode Program

```
1 package pekan3_2511531020;
2 import java.util.Scanner;
3
4
5 public class StackPostfix_2511531020 {
6     public static int postfixEvaluate_2511531020 (String expression) {
7         Stack<Integer> s = new Stack<Integer>();
8         Scanner input = new Scanner (expression);
9         while (input.hasNext()) {
10             if (input.hasNextInt()) { // an operand (integer)
11                 s.push(input.nextInt());
12             } else {
13                 String operator = input.next();
14                 int operand2 = s.pop();
15                 int operand1 = s.pop();
16                 if (operator.equals("+")) {
17                     s.push(operand1 + operand2);
18                 } else if (operator.equals("-")) {
19                     s.push(operand1 - operand2);
20                 } else if (operator.equals("*")) {
21                     s.push(operand1 * operand2);
22                 } else {
23                     s.push(operand1 / operand2);
24                 }
25             }
26         }
27         // input.close();
28         return s.pop();
29     }
30
31     public static void main(String[] args) {
32         System.out.println("hasil postfix= " + postfixEvaluate_2511531020("5 2 4 * + 7 -"));
33     }
34
35 }
```

Gambar 2.8 Class 2 “StackPostfix_2511531020”

4. Analisis Langkah

a. Tujuan Program

- Memahami cara kerja evaluasi ekspresi postfix menggunakan struktur data Stack.
- Melatih penggunaan operasi dasar Stack (push, pop) dalam konteks algoritma evaluasi ekspresi.
- Menunjukkan bagaimana Stack dapat digunakan untuk menyelesaikan operasi aritmatika secara sistematis.

b. Struktur Program

- Package: pekan3_2511531020 → menunjukkan program berada dalam paket tertentu.
- Class: StackPostfix_2511531020 → kelas utama yang berisi metode evaluasi postfix dan main.
- Method utama: public static void main(String[] args) → titik masuk eksekusi program.
- Method tambahan: postfixEvaluate_2511531020(String expression) → berfungsi untuk mengevaluasi ekspresi postfix.

c. Proses Eksekusi

- Membuat objek `Stack<Integer> s = new Stack<Integer>();` untuk menampung operand.
- Membaca ekspresi postfix menggunakan `Scanner input = new Scanner(expression);`.
- Loop `while (input.hasNext())` berjalan selama masih ada token:
 - Jika token berupa angka (`input.hasNextInt()`), angka tersebut dimasukkan ke stack dengan `push()`.
 - Jika token berupa operator (+, -, *, /), maka:
 - Ambil dua operand teratas dengan `pop()`.
 - Lakukan operasi sesuai operator.
 - Hasil operasi dimasukkan kembali ke stack.
- Setelah semua token diproses, hasil akhir diperoleh dengan `s.pop()`.

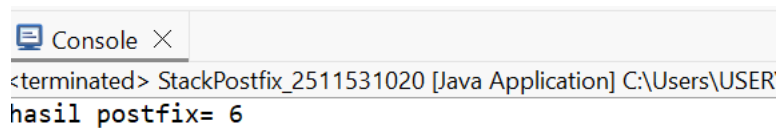
d. Menampilkan Hasil Evaluasi

- Pada main, program memanggil `postfixEvaluate_2511531020("5 2 4 * + 7 -")`.
- Ekspresi postfix ini berarti:
 - $5 + (2 * 4) - 7$
 - $= 5 + 8 - 7$
 - $= 6$.
- Output: hasil postfix= 6.

e. Konsep OOP dan Struktur Data

- Encapsulation: Stack digunakan sebagai objek, menyembunyikan detail implementasi internal.
- LIFO Principle: Operand terakhir yang masuk diproses lebih dulu saat operasi aritmatika dilakukan.
- Abstraksi: Programmer cukup menggunakan metode `push`, `pop`, tanpa perlu tahu detail internal Stack.
- Algoritma Evaluasi Postfix: Stack menjadi alat utama untuk memproses ekspresi postfix secara efisien.

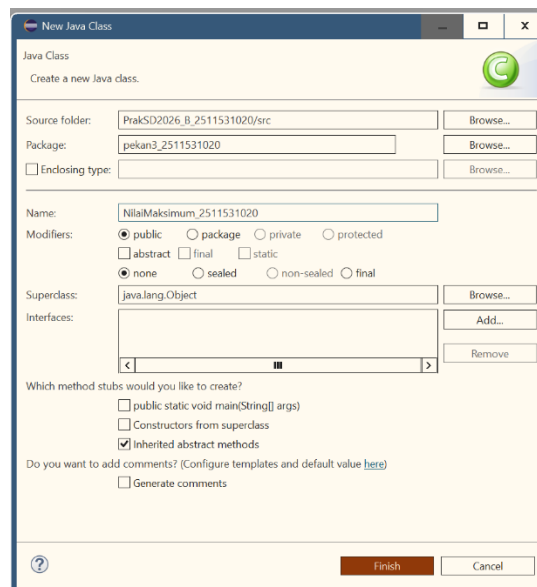
5. Output



Gambar 2.9 Output

2.2.5 Class 3 “NilaiMaksimum_2511531020”

1. Membuat package pekan3_2511531020 dan class NilaiMaksimum_2511531020



Gambar 2.10 Membuat Nama package dan class

2. Klik Finish

3. Kode Program

```
1 package pekan3_2511531020;
2
3 import java.util.Stack;
4
5 public class NilaiMaksimum_2511531020 {
6     public static int max_2511531020(Stack<Integer> s) {
7         Stack<Integer> backup = new Stack<Integer>();
8         int max_2511531020 = s.pop();
9         backup.push(max_2511531020);
10        while (!s.isEmpty()) {
11            int next = s.pop();
12            backup.push(next);
13            max_2511531020 = Math.max(max_2511531020, next);
14        }
15        while (!backup.isEmpty()) {
16            s.push(backup.pop());
17        }
18        return max_2511531020;
19    }
20
21    public static void main(String[] args) {
22        Stack<Integer> s = new Stack<Integer>();
23        s.push(70);
24        s.push(12);
25        s.push(20);
26        System.out.println("isi stack " + s);
27        System.out.println("Stack Teratas " + s.peek());
28        System.out.println("Nilai maksimum " + max_2511531020(s));
29    }
30 }
31 }
```

Gambar 2.11 Class 3 “NilaiMaksimum_2511531020”

4. Analisis Langkah

a. Tujuan Program

- Memahami cara mencari nilai maksimum dari elemen-elemen dalam sebuah Stack.
- Melatih penggunaan operasi dasar Stack (push, pop, peek) dalam konteks pencarian nilai.
- Menunjukkan bagaimana Stack dapat diproses tanpa kehilangan data dengan bantuan *backup stack*.

b. Struktur Program

- Package: pekan3_2511531020 → menunjukkan program berada dalam paket tertentu.
- Class: NilaiMaksimum_2511531020 → kelas utama yang berisi metode pencarian nilai maksimum dan main.
- Method utama: public static void main(String[] args) → titik masuk eksekusi program.

- Method tambahan: `max_2511531020(Stack<Integer> s)`
→ berfungsi untuk mencari nilai maksimum dari stack.

c. Proses Eksekusi

Input jam dari pengguna

- Program meminta pengguna memasukkan jam, menit, dan detik.
- Validasi dilakukan dengan `Jam_2511531020.isValid(h, m, s)`.
- Jika valid, objek jam dibuat; jika tidak, pengguna diminta mengulang.

d. Menampilkan data jam

Java:

```
System.out.println("Jam 1 (String)   : " + j1);
System.out.println("Jam 2 (String)   : " + j2);
System.out.println("Jam 1 dalam detik : " + j1.toSeconds());
System.out.println("Jam 2 dalam detik : " + j2.toSeconds());
```

- Memanfaatkan `toString()` untuk representasi jam.
- `toSeconds()` mengubah jam ke total detik.

e. Operasi relasional

Java:

```
int perbandingan = j1.compareTo(j2);
```

- Jika hasil > 0 → Jam 1 lebih lambat dari Jam 2.
- Jika hasil < 0 → Jam 1 lebih awal dari Jam 2.
- Jika hasil $= 0$ → Kedua jam sama.

f. Operasi aritmatika

Java:

```
System.out.println("Durasi   (j1   ke   j2)   :   "   +
Jam_2511531020.durasiDetik(j1, j2) + " detik");
Jam_2511531020 jNext = j1.nextSecond();
Jam_2511531020 jPrev = j1.prevSecond();
```


Jam_2511531020 jPlus = j1.plus(j2);

- durasiDetik(j1, j2) menghitung selisih waktu dalam detik.
- nextSecond() menambah 1 detik pada jam.
- prevSecond() mengurangi 1 detik pada jam.
- plus(j2) menambahkan jam kedua ke jam pertama.

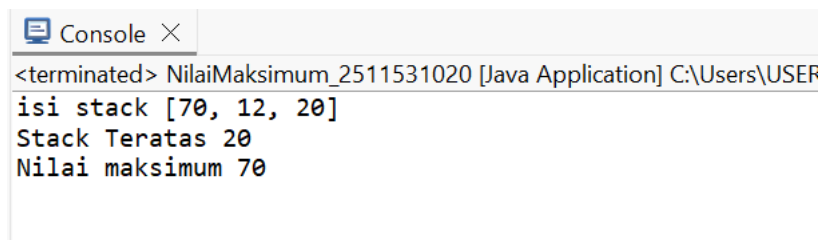
h. Akhir program

- Scanner ditutup dengan input.close().
- Program menampilkan pesan “Program Selesai.”

i. Analisis Konsep OOP

- Encapsulation: Input divalidasi melalui method isValid(), menjaga agar objek jam selalu konsisten.
- Polymorphism: Pemanggilan System.out.println(j1) memanfaatkan toString() untuk menampilkan jam.
- Inheritance/Interface: Method compareTo() menunjukkan kemungkinan implementasi Comparable.
- Static Method: durasiDetik() digunakan tanpa membuat objek baru, cukup memanggil langsung dari kelas.

5. Output

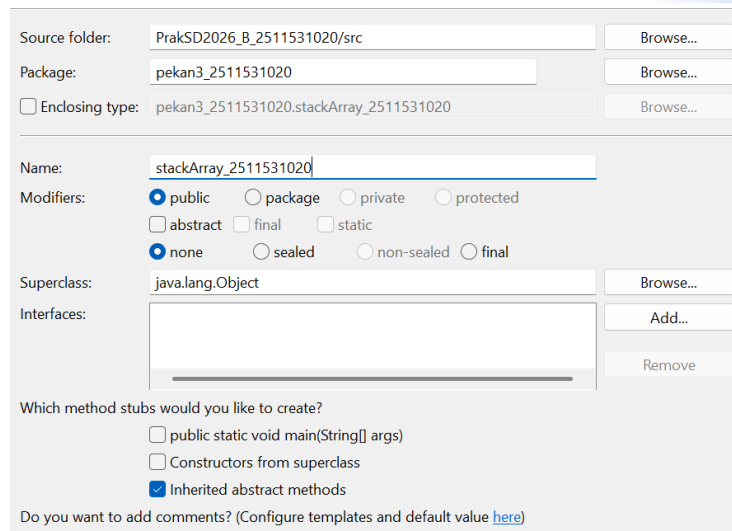
A screenshot of a Java IDE console window. The title bar says "Console X". The output text is: "<terminated> NilaiMaksimum_2511531020 [Java Application] C:\Users\USERF", "isi stack [70, 12, 20]", "Stack Teratas 20", and "Nilai maksimum 70".

```
<terminated> NilaiMaksimum_2511531020 [Java Application] C:\Users\USERF
isi stack [70, 12, 20]
Stack Teratas 20
Nilai maksimum 70
```

Gambar 2.12 Output

2.2.6 Class 4 “stackArray_2511531020”

1. Membuat package pekan3_2511531020 dan class stackArray_2511531020



Gambar 2.13 Membuat Nama package dan class

2. Klik Finish
3. Kode Program

```
1 package pekan3_2511531020;
2
3 public class stackArray_2511531020 {
4     static final int MAX = 1000;
5     int top;
6     int a[] = new int [MAX];
7     boolean isEmpty_2511531020()
8     {
9         return (top < 0);
10    }
11    stackArray_2511531020()
12    {
13        top = -1;
14    }
15    boolean push_2511531020 (int x)
16    {
17        if (top >= (MAX - 1)) {
18            System.out.println ("Stack Overflow");
19            return false;
20        }
21        else {
22            a[++top] = x;
23            System.out.println(x + " dimasukkan dalam stack");
24            return true;
25        }
26    }
27    int pop_2511531020 ()
28    {
29        if (top < 0) {
30            System.out.println ("Stack Underflow");
31            return 0;
32        }
33        else {
34            int x = a[top--];
35            return x;
```

```

36     }
37 }
38 int peek_2511531020()
39 {
40     if (top < 0) {
41         System.out.println("Stack Underflow");
42         return 0;
43     }
44     else {
45         int x = a[top];
46         return x;
47     }
48 }
49 void print_2511531020() {
50     for(int i = top; i > -1; i--) {
51         System.out.println(" " + a[i]);
52     }
53 }
54 }
55 }

```

Gambar 2.14 Class 3 “stackArray_2511531020”

4. Analisis Langkah

a. Tujuan Program

- Memahami implementasi Stack secara manual menggunakan array.
- Melatih penggunaan operasi dasar Stack (push, pop, peek) tanpa memanfaatkan class bawaan Java.
- Menunjukkan bagaimana prinsip *Last In First Out (LIFO)* dapat diterapkan dengan struktur array.

b. Struktur Program

- Package: pekan3_2511531020 → menunjukkan program berada dalam paket tertentu.
- Class: stackArray_2511531020 → kelas utama yang berisi definisi Stack berbasis array.
- Field:
 - static final int MAX = 1000; → kapasitas maksimum stack.
 - int top; → penunjuk posisi elemen teratas.
 - int a[] = new int[MAX]; → array untuk menyimpan elemen stack.
- Constructor: stackArray_2511531020() → inisialisasi top = -1 menandakan stack kosong.

c. Proses Eksekusi (Metode Utama)

- `isEmpty_2511531020()`: Mengecek apakah stack kosong (`top < 0`).
 - `push_2511531020(int x)`:
 - Jika `top` sudah mencapai batas (`MAX - 1`), tampilkan pesan *Stack Overflow*.
 - Jika belum penuh, tambahkan elemen ke array (`a[++top] = x`).
 - `pop_2511531020()`:
 - Jika stack kosong (`top < 0`), tampilkan pesan *Stack Underflow*.
 - Jika ada elemen, ambil elemen teratas (`a[top--]`).
 - `peek_2511531020()`:
 - Jika stack kosong, tampilkan pesan *Stack Underflow*.
 - Jika ada elemen, tampilkan elemen teratas (`a[top]`).
 - `print_2511531020()`: Menampilkan seluruh isi stack dari atas ke bawah.
- d. Contoh Eksekusi Program Misalnya dilakukan operasi:
- `push_2511531020(10)` → stack berisi [10].
 - `push_2511531020(20)` → stack berisi [10, 20].
 - `peek_2511531020()` → menampilkan 20.
 - `pop_2511531020()` → menghapus 20, stack kembali [10].
 - `print_2511531020()` → menampilkan isi stack: 10.
- e. Konsep OOP dan Struktur Data
- Encapsulation: Stack diimplementasikan sebagai objek dengan metode khusus.
 - Manual Implementation: Tidak menggunakan `java.util.Stack`, tetapi membuat logika sendiri dengan array.
 - LIFO Principle: Elemen terakhir yang masuk adalah yang pertama keluar.
 - Error Handling: Program menangani kondisi *Overflow* (stack penuh) dan *Underflow* (stack kosong).

- Abstraksi: Pengguna cukup memanggil metode push, pop, peek, tanpa perlu tahu detail array internal.

2.2.7 Class 5 “stackArrayDriver_2511531020”

1. Membuat package pekan3_2511531020 dan class stackArrayDriver_2511531020

Source folder:

Package:

☐ Enclosing type:

Name:

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static
☒ none ☐ sealed ☐ non-sealed ☐ final

Superclass:

Interfaces:

Which method stubs would you like to create?
☐ public static void main(String[] args)
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))
☐ Generate comments

Gambar 2.15 Membuat Nama package dan class

2. Klik Finish
3. Kode Program

```

1 package pekan3_2511531020;
2
3 public class StackArrayDriver_2511531020 {
4     public static void main (String[] args) {
5         stackArray_2511531020 s = new stackArray_2511531020();
6         s.push_2511531020 (10);
7         s.push_2511531020(20);
8         s.push_2511531020(30);
9         System.out.println(s.pop_2511531020 () + " dikeluarkan dari stack");
10        System.out.println("Elemen teratas adalah :" + s.peek_2511531020());
11        System.out.println("Elemen pada stack :");
12        s.print_2511531020();
13    }
14 }
15
16 }

```

Gambar 2.16 Class 3 “stackArrayDriver_2511531020”

4. Analisis Langkah

a. Tujuan Program

- Menguji implementasi Stack berbasis array (stackArray_2511531020).
- Melatih penggunaan operasi dasar Stack: push, pop, peek, dan print.
- Menunjukkan prinsip *Last In First Out (LIFO)* dalam eksekusi nyata.

b. Struktur Program

- Package: pekan3_2511531020 → menunjukkan program berada dalam paket tertentu.
- Class: StackArrayDriver_2511531020 → kelas driver yang berfungsi menjalankan dan menguji class stackArray_2511531020.
- Method utama: public static void main(String[] args) → titik masuk eksekusi program.

c. Proses Eksekusi

- Membuat objek stackArray_2511531020 s = new stackArray_2511531020();.
- Menambahkan elemen ke stack dengan:
 - s.push_2511531020(10); → stack berisi [10].
 - s.push_2511531020(20); → stack berisi [10, 20].
 - s.push_2511531020(30); → stack berisi [10, 20, 30].
- Menghapus elemen teratas dengan s.pop_2511531020(); → elemen 30 keluar.
 - Output: 30 dikeluarkan dari stack.
 - Isi stack sekarang [10, 20].
- Melihat elemen teratas dengan s.peek_2511531020(); → hasilnya 20.
 - Output: Elemen teratas adalah :20.

- Menampilkan seluruh isi stack dengan `s.print_2511531020();`.
 - Output:

Code

Elemen pada stack :

20

10

d. Konsep OOP dan Struktur Data

- Encapsulation: Stack diimplementasikan sebagai objek dengan metode khusus (push, pop, peek, print).
- Driver Class: Digunakan untuk menguji fungsionalitas class utama.
- LIFO Principle: Elemen terakhir yang masuk (30) adalah yang pertama keluar saat pop().
- Manual Implementation: Stack berbasis array memberikan pemahaman mekanisme internal tanpa bergantung pada `java.util.Stack`.
- Error Handling: Class `stackArray_2511531020` sudah menyiapkan kondisi *Overflow* dan *Underflow*.

5. Output

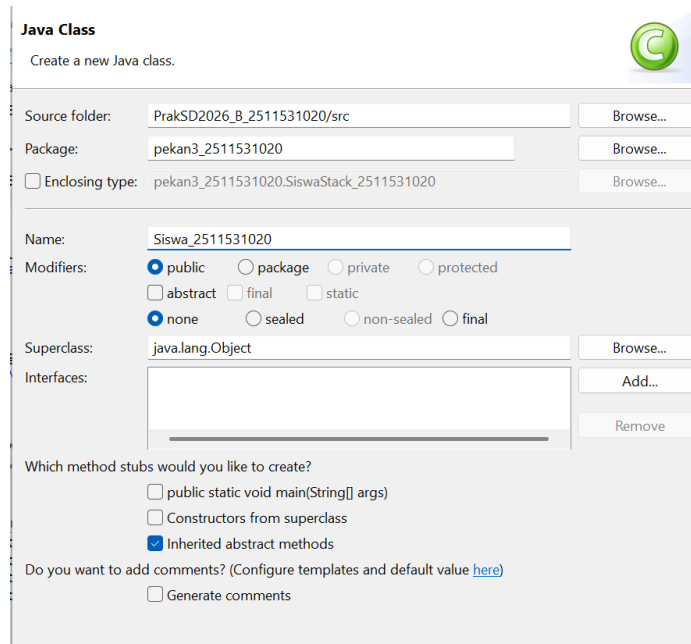


```
<terminated> StackArrayDriver_2511531020 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.openjdk.
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah :20
Elemen pada stack :|
20
10
```

Gambar 2.17 Output

2.2.8 Class 6 “Siswa_2511531020”

1. Membuat package pekan3_2511531020 dan class Siswa_2511531020



Java Class
Create a new Java class.

Source folder: Browse...

Package: Browse...

☐ Enclosing type: Browse...

Name:

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static
☒ none ☐ sealed ☐ non-sealed ☐ final

Superclass: Browse...

Interfaces: Add... Remove

Which method stubs would you like to create?
☐ public static void main(String[] args)
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))
☐ Generate comments

Gambar 2.18 Membuat Nama package dan class

3 Klik Finish

4 Kode Program

```
1 package pekan3_2511531020;  
2  
3 import java.util.ArrayList;  
4  
5 class Siswa_2511531020 {  
6     String nama;  
7     int nim;  
8  
9     public Siswa_2511531020(String nama, int nim) {  
10         this.nama = nama;  
11         this.nim = nim;  
12     }  
13  
14     @Override  
15     public String toString() {  
16         return "Nim: " + nim + ", Nama: " + nama;  
17     }  
18 }  
19  
20 public class SiswaStack_2511531020 {  
21     private ArrayList<Siswa_2511531020> stack;  
22  
23     public SiswaStack_2511531020() {  
24         stack = new ArrayList<>();  
25     }  
26  
27     public void push_2511531020(Siswa_2511531020 mhs) {  
28         stack.add(mhs);  
29     }  
30  
31     public Siswa_2511531020 pop_2511531020() {  
32         if (!isEmpty_2511531020()) {  
33             return stack.remove(stack.size() - 1);  
34         }  
35         return null;  
36     }  
37 }
```



```

36 }
37
38 public Siswa_2511531020 peek_2511531020() {
39     if (!isEmpty_2511531020()) {
40         return stack.get(stack.size() - 1);
41     }
42     return null;
43 }
44
45 public boolean isEmpty_2511531020() {
46     return stack.isEmpty();
47 }
48
49 public void tampilkanSiswa_2511531020() {
50     for (int i = stack.size() - 1; i >= 0; i--) {
51         System.out.println(stack.get(i));
52     }
53 }
54
55 public static void main(String[] args) {
56     SiswaStack_2511531020 studentStack = new SiswaStack_2511531020();
57
58     Siswa_2511531020 mhs1 = new Siswa_2511531020("Ali", 1);
59     Siswa_2511531020 mhs2 = new Siswa_2511531020("Boby", 2);
60     Siswa_2511531020 mhs3 = new Siswa_2511531020("Charles", 3);
61
62     studentStack.push_2511531020(mhs1);
63     studentStack.push_2511531020(mhs2);
64     studentStack.push_2511531020(mhs3);
65
66     System.out.println("Siswa di dalam stack:");
67     studentStack.tampilkanSiswa_2511531020();
68
69     System.out.println("Siswa teratas: " + studentStack.peek_2511531020());
70     System.out.println("Mengeluarkan siswa: " + studentStack.pop_2511531020());
71
72     System.out.println("Setelah pop:");
73     studentStack.tampilkanSiswa_2511531020();
74 }
75 }

```

Gambar 2.19 Class 3 “Siswa_2511531020”

5 Analisis Langkah

a. Tujuan Program

- Memahami implementasi Stack menggunakan ArrayList untuk menyimpan objek bertipe Siswa.
- Melatih penggunaan operasi dasar Stack (push, pop, peek, isEmpty) dalam konteks data mahasiswa/siswa.
- Menunjukkan penerapan prinsip *Last In First Out (LIFO)* pada objek buatan sendiri.

b. Struktur Program

- Package: pekan3_2511531020 → menunjukkan program berada dalam paket tertentu.
- Class:
 - Siswa_2511531020 → merepresentasikan entitas siswa dengan atribut nama dan nim.

- SiswaStack_2511531020 → kelas utama yang mengimplementasikan stack berbasis ArrayList<Siswa_2511531020>.
- Method utama: public static void main(String[] args) → titik masuk eksekusi program.
- Method tambahan dalam SiswaStack_2511531020:
 - push_2511531020() → menambahkan siswa ke stack.
 - pop_2511531020() → menghapus siswa teratas dari stack.
 - peek_2511531020() → melihat siswa teratas tanpa menghapus.
 - isEmpty_2511531020() → mengecek apakah stack kosong.
 - tampilkanSiswa_2511531020() → menampilkan seluruh isi stack dari atas ke bawah.

c. Proses Eksekusi

- Membuat objek SiswaStack_2511531020 studentStack = new SiswaStack_2511531020();
- Membuat tiga objek siswa:
 - Ali (nim=1)
 - Bobby (nim=2)
 - Charles (nim=3)
- Menambahkan ketiga siswa ke stack dengan push.
 - Isi stack: [Ali, Bobby, Charles] (Charles berada di atas).
- Menampilkan seluruh isi stack dengan tampilkanSiswa_2511531020().
 - Output:

Code

Nim: 3, Nama: Charles

Nim: 2, Nama: Bobby

Nim: 1, Nama: Ali

- Melihat siswa teratas dengan peek_2511531020().
 - Output: Siswa teratas: Nim: 3, Nama: Charles.
- Menghapus siswa teratas dengan pop_2511531020().

- Output: Mengeluarkan siswa: Nim: 3, Nama: Charles.
- Isi stack sekarang [Ali, Bobby].
- Menampilkan isi stack setelah pop.
 - Output:

Code

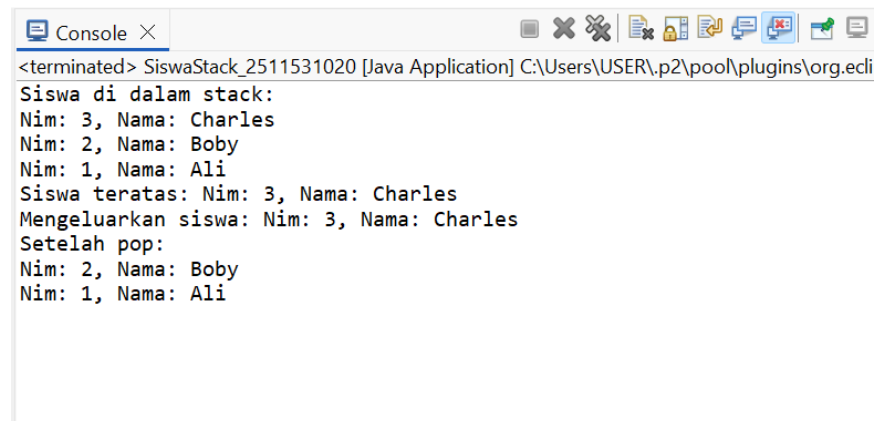
Nim: 2, Nama: Bobby

Nim: 1, Nama: Ali

d. Konsep OOP dan Struktur Data

- Encapsulation: Stack diimplementasikan sebagai objek dengan metode khusus.
- LIFO Principle: Siswa terakhir yang masuk (Charles) adalah yang pertama keluar saat pop().
- Abstraksi: Programmer cukup menggunakan metode push, pop, peek, tanpa perlu tahu detail internal ArrayList.
- Custom Object Handling: Stack tidak hanya menyimpan tipe primitif, tetapi juga objek buatan sendiri (Siswa).
- Flexibilitas Output: Data siswa dapat ditampilkan sekaligus atau satu per satu sesuai kebutuhan.

6 Output



```

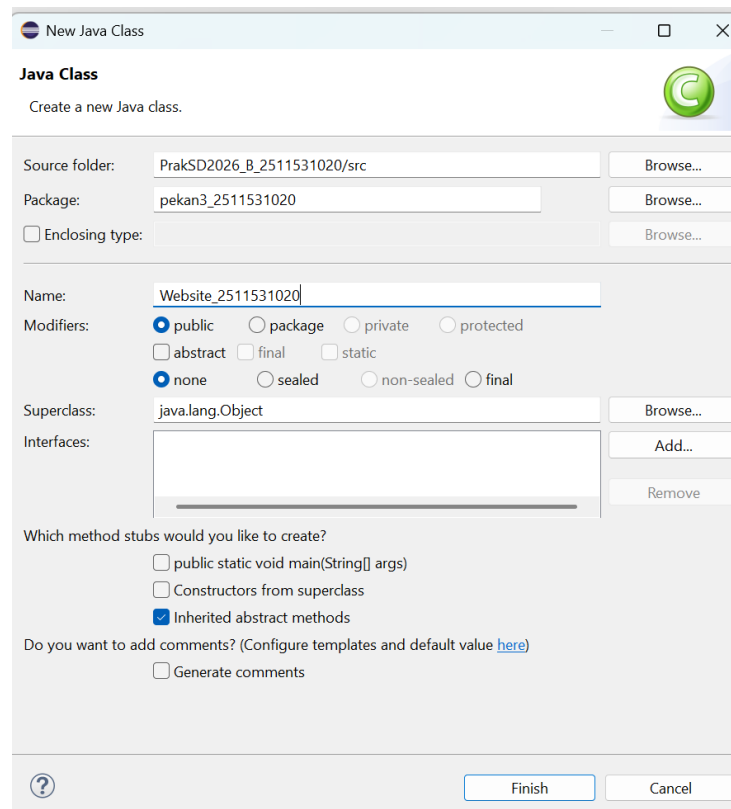
<terminated> SiswaStack_2511531020 [Java Application] C:\Users\USER\p2\pool\plugins\org.ecli
Siswa di dalam stack:
Nim: 3, Nama: Charles
Nim: 2, Nama: Bobby
Nim: 1, Nama: Ali
Siswa teratas: Nim: 3, Nama: Charles
Mengeluarkan siswa: Nim: 3, Nama: Charles
Setelah pop:
Nim: 2, Nama: Bobby
Nim: 1, Nama: Ali
  
```

Gambar 2.20 Output

2.3 Tugas Praktikum

2.3.1 Class Website_2511531020

1. Membuat Nama package pekan3_2511531020 dan class Website_2511531020



Gambar 2.21 Membuat Nama package dan class

2. Kode Program

```
1 package pekan3_2511531020;
2
3 public class Website_2511531020 {
4     private String judul_1020;
5     private String url_1020;
6
7     public Website_2511531020(String judul_1020, String url_1020) {
8         this.judul_1020 = judul_1020;
9         this.url_1020 = url_1020;
10    }
11
12    public String getJudul_1020() {
13        return judul_1020;
14    }
15
16    public void setJudul_1020(String judul_1020) {
17        this.judul_1020 = judul_1020;
18    }
19
20    public String getUrl_1020() {
21        return url_1020;
22    }
23
24    public void setUrl_1020(String url_1020) {
25        this.url_1020 = url_1020;
26    }
27
28    @Override
29    public String toString() {
30        return "Judul: " + judul_1020 + ", URL: " + url_1020;
31    }
32 }
33
```

Gambar 2.22 Class Website_2511531020

3. Analisis Langkah

a. Tujuan Program

- Merepresentasikan data sebuah website dalam bentuk objek.
- Menyediakan atribut penting (judul dan URL) yang dapat diakses dan dimodifikasi.
- Menjadi *blueprint* (ADT) yang akan digunakan oleh kelas driver Browser_2511531020 untuk simulasi browser history.

b. Struktur Program

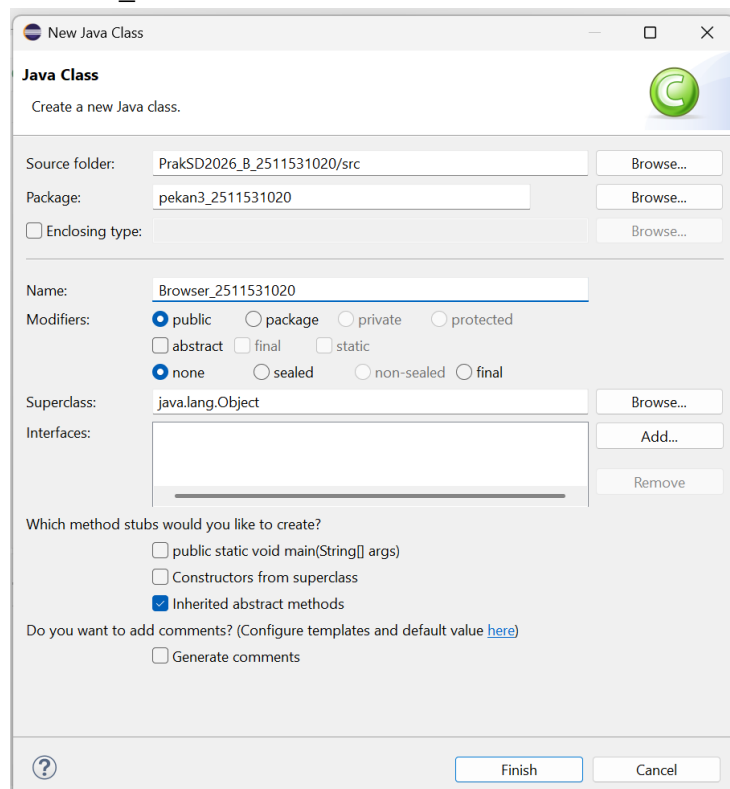
- Package: pekan3_2511531020 → menunjukkan program berada dalam paket tertentu.
- Class: Website_2511531020 → kelas ADT yang menyimpan data website.
- Atribut:
 - judul_1020 → menyimpan judul website.

- url_1020 → menyimpan alamat URL website.
 - Constructor: Website_2511531020(String judul_1020, String url_1020) → menginisialisasi objek dengan judul dan URL.
 - Getter (Selektor): getJudul_1020(), getUrl_1020() → mengambil nilai atribut.
 - Setter (Mutator): setJudul_1020(), setUrl_1020() → mengubah nilai atribut.
 - Method toString(): menampilkan representasi string dari objek website (judul dan URL).
- c. Proses Eksekusi
- Saat objek baru dibuat dengan new Website_2511531020("SSO UNAND", "https://sso.unand.ac.id/login"), constructor akan menyimpan judul dan URL ke atribut.
 - Jika dipanggil getJudul_1020(), maka akan mengembalikan "SSO UNAND".
 - Jika dipanggil getUrl_1020(), maka akan mengembalikan "https://sso.unand.ac.id/login".
 - Jika dipanggil toString(), maka akan menampilkan:
Code
Judul: SSO UNAND, URL: https://sso.unand.ac.id/login
 - Atribut dapat diubah dengan setJudul_1020("Portal UNAND") atau setUrl_1020("https://portal.unand.ac.id").
- d. Konsep OOP dan Struktur Data
- Encapsulation: atribut judul_1020 dan url_1020 disembunyikan (private), hanya bisa diakses melalui getter dan setter.
 - Abstraksi: kelas Website_2511531020 menyederhanakan data website menjadi objek dengan atribut dan metode yang jelas.

- Reusability: objek website dapat digunakan berulang kali dalam stack untuk simulasi browser history.
- Data Representation: toString() memberikan cara mudah untuk menampilkan isi objek ke konsol.

2.3.2 Class Browser_2511531020

1. Membuat Nama package pekan3_2511531020 dan class Browser_2511531020



Gambar 2.23 Membuat Nama package dan class

2. Kode Program

```
1 package pekan3_2511531020;
2
3 import java.util.Scanner;
4 import java.util.Stack;
5
6 public class Browser_2511531020 {
7     public static void main(String[] args) {
8         Stack<Website_2511531020> history = new Stack<>();
9         Scanner sc = new Scanner(System.in);
10        int pilihan;
11
12        do {
13            System.out.println("== Browser History NIM: 2511531020 ==");
14            System.out.println("1. Kunjungi Website (Push)");
15            System.out.println("2. Tombol Back (Pop)");
16            System.out.println("3. Lihat Halaman Aktif (Peek)");
17            System.out.println("4. Cek Status History");
18            System.out.println("5. Keluar");
19            System.out.print("Pilihan: ");
20            pilihan = sc.nextInt();
21            sc.nextLine();
22
23            switch (pilihan) {
24                case 1:
25                    System.out.print("Masukkan Judul: ");
26                    String judul = sc.nextLine();
27                    System.out.print("Masukkan URL: ");
28                    String url = sc.nextLine();
29                    history.push(new Website_2511531020(judul, url));
30                    System.out.println("Berhasil mengunjungi halaman!\n");
31                    break;
32                case 2:
33                    if (!history.isEmpty()) {
34                        Website_2511531020 removed = history.pop();
35                        System.out.println("Kembali dari halaman: " + removed.getJudul_1020() + "\n");
36
37                    } else {
38                        System.out.println("History kosong, tidak ada halaman untuk kembali.\n");
39                    }
40                    break;
41                case 3:
42                    if (!history.isEmpty()) {
43                        System.out.println("Halaman aktif: " + history.peek() + "\n");
44                    } else {
45                        System.out.println("Tidak ada halaman aktif.\n");
46                    }
47                    break;
48                case 4:
49                    System.out.println("Jumlah riwayat: " + history.size());
50                    System.out.println("Apakah history kosong? " + history.isEmpty() + "\n");
51                    break;
52                case 5:
53                    System.out.println("Program selesai.");
54                    break;
55                default:
56                    System.out.println("Pilihan tidak valid.\n");
57            }
58        } while (pilihan != 5);
59        sc.close();
60    }
61 }
62 }
```

Gambar 2.24 Class Browser_2511531020

3. Analisis Langkah

a. Tujuan Program

- Mengimplementasikan simulasi Browser History menggunakan struktur data Stack.
- Menyediakan menu interaktif bagi pengguna untuk melakukan operasi push, pop, peek, dan pengecekan status history.

- Menunjukkan penerapan prinsip *Last In First Out (LIFO)* dalam konteks nyata (riwayat halaman web).
- b. Struktur Program
- Package: `pekan3_2511531020` → menunjukkan program berada dalam paket tertentu.
 - Class: `Browser_2511531020` → kelas driver yang mengatur logika simulasi browser history.
 - Method utama: `public static void main(String[] args)` → titik masuk eksekusi program.
 - Objek Stack: `Stack<Website_2511531020> history` → menyimpan riwayat website yang dikunjungi.
 - Scanner: digunakan untuk membaca input dari pengguna.
 - Menu interaktif: terdiri dari pilihan 1–5 (Push, Pop, Peek, Cek Status, Keluar).
- c. Proses Eksekusi
- Program menampilkan menu utama dengan pilihan operasi Stack.
 - Pilihan 1 (Push):
 - Pengguna memasukkan judul dan URL website.
 - Objek `Website_2511531020` baru dibuat dan dimasukkan ke stack.
 - Output: “Berhasil mengunjungi halaman!”.
 - Pilihan 2 (Pop):
 - Jika stack tidak kosong, elemen teratas dihapus dan ditampilkan judulnya.
 - Jika kosong, tampilkan pesan “History kosong, tidak ada halaman untuk kembali.”
 - Pilihan 3 (Peek):
 - Jika stack tidak kosong, tampilkan detail website teratas tanpa menghapusnya.
 - Jika kosong, tampilkan pesan “Tidak ada halaman aktif.”

- Pilihan 4 (Cek Status History):
 - Menampilkan jumlah riwayat yang tersimpan (`history.size()`).
 - Menampilkan status apakah stack kosong (`history.isEmpty()`).
- Pilihan 5 (Keluar):
 - Program berhenti dengan pesan “Program selesai.”

d. Contoh Output

Code:

=== Browser History NIM: 2511531020 ===

1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar

Pilihan: 1

Masukkan Judul: SSO UNAND

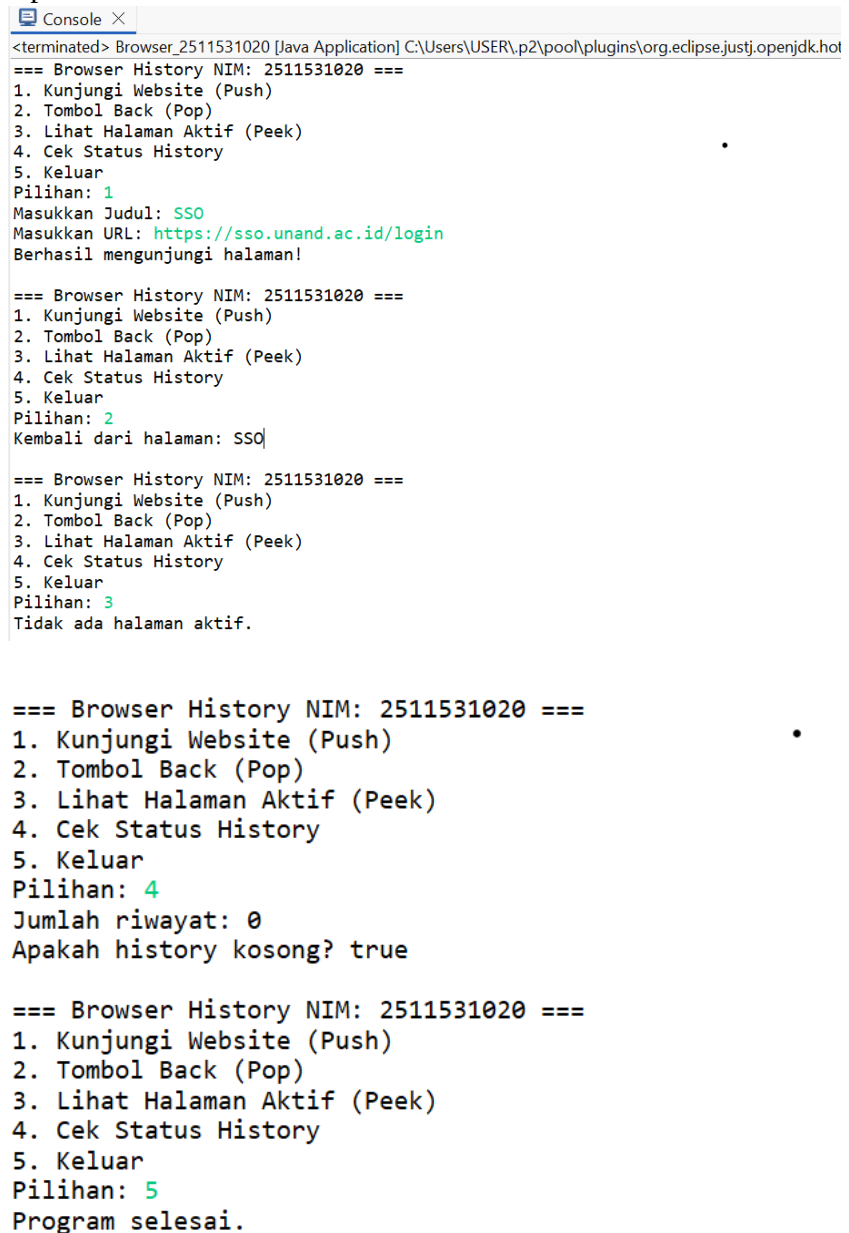
Masukkan URL: <https://sso.unand.ac.id/login>

Berhasil mengunjungi halaman!

e. Konsep OOP dan Struktur Data

- Encapsulation: objek `Website_2511531020` digunakan sebagai elemen stack dengan atribut privat.
- LIFO Principle: halaman terakhir yang dikunjungi adalah yang pertama keluar saat tombol Back ditekan.
- Abstraksi: pengguna hanya berinteraksi dengan menu, tanpa perlu tahu detail implementasi stack.
- Error Handling: program menangani kondisi stack kosong dengan aman (tidak error).
- Driver Role: kelas driver berfungsi sebagai pengendali logika, sementara kelas ADT hanya sebagai model data.

4. Output



```
Console X
<terminated> Browser_2511531020 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.openjdk.hot
=== Browser History NIM: 2511531020 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 1
Masukkan Judul: SSO
Masukkan URL: https://sso.unand.ac.id/login
Berhasil mengunjungi halaman!

=== Browser History NIM: 2511531020 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 2
Kembali dari halaman: SSO

=== Browser History NIM: 2511531020 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 3
Tidak ada halaman aktif.

=== Browser History NIM: 2511531020 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 4
Jumlah riwayat: 0
Apakah history kosong? true

=== Browser History NIM: 2511531020 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 5
Program selesai.
```

Gambar 2.25 Output

BAB III

KESIMPULAN

Kesimpulan dari seluruh praktikum bertema Stack adalah bahwa mahasiswa telah berhasil memahami dan mengimplementasikan prinsip dasar struktur data tumpukan dengan konsep *Last In First Out (LIFO)*. Melalui berbagai program seperti latihanStack, StackPostfix, NilaiMaksimum, stackArray, StackArrayDriver, hingga SiswaStack, terlihat bahwa operasi fundamental seperti push, pop, peek, dan isEmpty dapat diterapkan secara konsisten baik menggunakan class bawaan Java (java.util.Stack), implementasi manual dengan array, maupun dengan ArrayList untuk menyimpan objek buatan sendiri. Praktikum ini juga menunjukkan penerapan nyata stack dalam kehidupan sehari-hari, misalnya simulasi browser history yang meniru tombol *Back* pada peramban web, serta evaluasi ekspresi postfix yang digunakan dalam algoritma matematika. Selain itu, konsep OOP seperti enkapsulasi, abstraksi, dan reusabilitas telah diterapkan dengan baik, sehingga mahasiswa tidak hanya memahami teori tetapi juga mampu mengaplikasikannya dalam kode yang rapi, terstruktur, dan sesuai aturan penamaan. Secara keseluruhan, praktikum ini membuktikan bahwa stack adalah struktur data fundamental yang penting, fleksibel, dan memiliki banyak aplikasi praktis, serta menjadi fondasi untuk mempelajari algoritma dan struktur data yang lebih kompleks di masa depan.

DAFTAR PUSTAKA

- [1] Oracle, “Class Stack (Java Platform SE 8),” *Oracle Documentation*, 2015. [Online]. Available: <https://docs.oracle.com/javase/8/docs/api/java/util/Stack.html> (docs.oracle.com in Bing)
- [2] Oracle, “Class ArrayList (Java Platform SE 8),” *Oracle Documentation*, 2015. [Online]. Available: <https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html> (docs.oracle.com in Bing)
- [3] C. Horstmann, *Big Java: Early Objects*, 7th ed. Hoboken, NJ: Wiley, 2019.
- [4] H. Schildt, *Java: The Complete Reference*, 12th ed. New York, NY: McGraw-Hill, 2021.
- [5] P. Deitel and H. Deitel, *Java How to Program*, 11th ed. Upper Saddle River, NJ: Pearson, 2017.
- [6] R. Lafore, *Data Structures and Algorithms in Java*, 2nd ed. Indianapolis, IN: Sams Publishing, 2002.